

Logique et raisonnement

CM2 : Modélisation en logique propositionnelle

Catalin Dima, UPEC

2021

Vérification de la satisfaisabilité

Problème SAT

Étant donné une formule φ , décider si φ est satisfaisable.

Utilité :

- ▶ Résolution des contraintes logiques (vu en CM1).
- ▶ Raisonnement "sémantique" :
 - ▶ Une formule G peut être déduite à partir des prémisses $F_1, \dots, F_n$ si $(F_1 \wedge \dots \wedge F_n) \rightarrow G$ est une tautologie,
 - ▶ ... ce qui est équivalent à tester que $\neg((F_1 \wedge \dots \wedge F_n) \rightarrow G)$ est **non-satisfaisable**.
- ▶ Bcp d'autres problèmes algorithmiques se ramènent à des problèmes de satisfaisabilité :
 - ▶ Problèmes algorithmiques de décision (exemple : graphes).
 - ▶ Conception des circuits logiques.
 - ▶ Planification.
 - ▶ Cryptographie.
 - ▶ Vérification de correction d'algorithme.
- ▶ Avantage : Implémentation assez simple : tester toutes les valuations des variables propositionnelles.
- ▶ Désavantage : temps exécution exponentiel (en nombre de variables propositionnelles).

Logiciel ToulST (Toulouse Integrated Satisfiability Tool)

- ▶ Logiciel développé à l'Institut de Recherche en Informatique de Toulouse : <https://www.irit.fr/touist/>
- ▶ Vérification de satisfaisabilité d'un "programme logique" décrit en une extension de la logique propositionnelle.
- ▶ Opérateurs étendus : $\bigwedge_{i \in [1..n]} p_i$, *atleast*, *atmost* etc...
- ▶ Paramètres.
- ▶ Implémenté en Java, logiciel fourni sous la forme d'un fichier *jar*.
- ▶ Affichage "mathématique" des formules (interface avec LaTeX, logiciel de rédaction d'articles mathématiques).

Un programme ToulST

Superman existe ?

Si Superman voulait et pouvait prévenir le mal, il le ferait : $F_1 \equiv (pp \wedge v \rightarrow p)$

Si Superman ne pouvait prévenir le mal, il serait impuissant ; s'il ne voulait pas prévenir le mal, il serait malveillant : $F_2 \equiv (\neg pp \rightarrow i) \wedge (\neg v \rightarrow m)$.

Superman ne prévient pas le mal : $F_3 \equiv \neg p$.

Si Superman existe, il n'est ni impuissant, ni malveillant : $F_4 \equiv e \rightarrow (\neg i \wedge \neg m)$.

Par conséquent, Superman n'existe pas : $G \equiv \neg e$.

Programme ToulST :

```
not ( (((pp and v) => p)
      and
      ((not pp => i) and (not v => m))
      and
      (not p)
      and
      (e => (not i and not m)))
=> not e)
```

Attention !

- ▶ On essaye de prouver que $(F_1 \wedge F_2 \wedge F_3 \wedge F_4) \rightarrow G$ est une tautologie
- ▶ ... ce qui est équivalent avec le fait de prouver que $\neg((F_1 \wedge F_2 \wedge F_3 \wedge F_4 \rightarrow G))$ n'est pas satisfaisable !

ToulST simplifié

- ▶ Pas besoin de déclarer les propositions atomiques !
 - ▶ Les propositions atomiques sont identifiées et parsées par la grammaire ToulST.
 - ▶ Chaque nouvelle chaîne de caractères identifiée dans une formule est considérée comme une nouvelle proposition atomique.
- ▶ Opérateurs : `not`, `xor`, `and`, `or`, `=>`, `<=>`.
- ▶ Priorité : dans l'ordre ci-dessus, sauf pour `=>`/`<=>` qui ont la même priorité et sont évalués de **droite à gauche**.
- ▶ Un fichier entier définit une seule formule.
- ▶ Retour à la ligne $\equiv$ opérateur `and` !
- ▶ Satisfaisabilité vérifiée : affichage d'une valuation des atomes qui satisfont la formule
 - ▶ `Next/Previous` (valuation).
 - ▶ Possibilité d'exporter chaque valuation.
- ▶ Satisfaisabilité non-vérifiée – message d'erreur.
 - ▶ Cela devrait être notre cas !

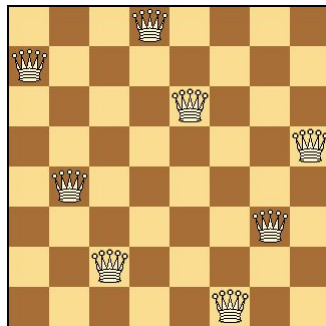
ToulST niveau intermédiaire et problèmes de satisfaisabilité paramétrés

Et si on devait résoudre une énigme logique / un problème de résolution de contrainte paramétré ?

Problème des N reines

Placer sur un échiquier $N \times N$, N reines de sorte qu'aucune reine n'attaque une autre.

- ▶ Une reine par colonne.
- ▶ Une reine par ligne.
- ▶ Chaque diagonale (ascendante ou descendante) ne peut avoir plus d'une reine.
- ▶ À droite : exemple de solution pour 8 reines.
- ▶ Cette fois-ci ToulST sera utilisé pour tester la satisfaisabilité !



Modélisation des N reines

Quelles sont les propositions atomiques ?

- ▶ Pour chaque ligne i et colonne j , une proposition atomique r_{ij} .
- ▶ $r_{ij} = 1$ si et seulement si on a une reine en position (i, j) .

Quelles sont les contraintes ?

- ▶ Une seule reine par colonne
- ▶ Une seule reine par ligne.
- ▶ Au maximum une reine par diagonale.
- ▶ Toutes ces contraintes doivent être transformées en **formules propositionnelles**.
- ▶ Et on cherche la satisfaisabilité de la **conjonction** de toutes ces formules !

Modélisation des N reines

► Une seule reine par colonne : $\bigwedge_{0 \leq i < N} \bigvee_{0 \leq j < N} \left(r_{ij} \wedge \bigwedge_{k \neq j} \neg r_{ik} \right)$

► Une seule reine par ligne : $\bigwedge_{0 \leq i < N} \bigvee_{0 \leq j < N} \left(r_{ij} \wedge \bigwedge_{k \neq i} \neg r_{kj} \right)$

► Maximum une reine sur chaque diagonale ascendante :

► Pour chaque i, j , si on a une reine en position (i, j) alors on doit pas avoir une reine sur une position (k, l) qui est sur la même diagonale ascendante :

$$\bigwedge_{0 \leq i, j < N} r_{ij} \rightarrow \bigwedge_{\substack{0 \leq k, l < N \\ i-k=j-l, i \neq k}} \neg r_{kl}$$

► Maximum une reine sur chaque diagonale descendante :

$$\bigwedge_{0 \leq i, j < N} r_{ij} \rightarrow \bigwedge_{\substack{0 \leq k, l < N \\ i+k=j+l, i \neq k}} \neg r_{kl}$$

► Il est essentiel de rajouter la contrainte $i \neq k$ pour qu'on évite de faire apparaître $\neg r_{ij}$ dans la dernière partie des formules, sinon on obtient des formules non-satisfaisables !

Variables et opérateurs étendus en ToulST

- ▶ "Variable" ToulST = paramètre.
 - ▶ Peuvent être utilisées avec des déclarations globales ou en tant qu'indices dans une formule (voir ci-dessous).
- ▶ Propositions atomiques indexées : $p(0)$, $q(2)$ etc.
 - ▶ On peut utiliser des variables pour indexer des atomes : $p(\$i)$.
 - ▶ On peut même avoir des doubles indices : $p(\$i, \$j)$.
- ▶ bigand équivalent d'une **conjonction paramétrée** :

```
$N = 8  
bigand $i in [0..$N] :  
    p($i)  
end
```

Équivalent à $\bigwedge_{\$i \in [0..\$N]} p(\$i) \equiv p(0) \wedge p(1) \wedge \dots \wedge p(\$N)$.

- ▶ On voit l'intérêt d'utiliser des variables ToulST en tant qu'indices !

Variables et opérateurs étendus en ToulST

- ▶ On peut même avoir des indices double (ou multiples) dans un `bigand` :

```
bigand $i,$j in [0..$N],[0..$M] :  
  p($i)  
end
```

- ▶ Et encore plus intéressant, limiter les valeurs d'indices selon certaines **conditions** :

```
bigand $i,$j in [0..$N],[0..$M] when $i != $j :  
  p($i)  
end
```

- ▶ Similaire : `bigor`.

- ▶ Pas de `bigxor` !

- ▶ `exact(1,p([0..$N]))` est équivalent à $\bigvee_{i \in [0..N]} \left(p(i) \wedge \bigwedge_{j \neq i} p(j) \right)$

Les N reines en ToulST

$N = 8$

```
bigand $i in [0..$N-1]:  
    exact(1, r($i, [0..$N-1]))
```

end

```
bigand $j in [0..$N-1]:  
    exact(1, r([0..$N-1], $j))
```

end

```
bigand $i, $j in [0..$N-1], [0..$N-1]:  
    r($i, $j) =>  
        bigand $k, $l in [0..$N-1], [0..$N-1] when ($i-$k==$j-$l and $i!=$k):  
            not r($k, $l)
```

end

```
bigand $i, $j in [0..$N-1], [0..$N-1]:  
    r($i, $j) =>  
        bigand $k, $l in [0..$N-1], [0..$N-1] when ($i+$j==$k+$l and $i!=$k):  
            not r($k, $l)
```

end

- ▶ Pas d'opérateur entre les lignes = opérateur **and** implicite !
- ▶ Remarquer la possibilité d'effectuer des opérations arithmétiques et des comparaisons sur les variables ToulST.

Encore des opérateurs ToulST

- ▶ Autres variantes d'`exact` :

`exact(2,q([0..$N]))`

- ▶ Veut dire **exactement 2 valeurs "positives" parmi $[q(0), \dots, q(\$N)]$ – les autres doivent être "négatives"**.
- ▶ Pour $\$N = 4$ ça ferait :

$$\begin{aligned} & (q(0) \wedge q(1) \wedge \neg q(2) \wedge \neg q(3)) \vee (q(0) \wedge \neg q(1) \wedge q(2) \wedge \neg q(3)) \vee \\ & (q(0) \wedge \neg q(1) \wedge \neg q(2) \wedge q(3)) \vee (\neg q(0) \wedge q(1) \wedge q(2) \wedge \neg q(3)) \vee \\ & (\neg q(0) \wedge q(1) \wedge \neg q(2) \wedge q(3)) \vee (\neg q(0) \wedge \neg q(1) \wedge q(2) \wedge q(3)) \end{aligned}$$

- ▶ `atmost` et `atleast` dont le nom est suggestif et qui possèdent la même syntaxe :

```
atmost(2,p([0..$N])) ;; au moins 2 parmi toute la liste [p(0)...p($N)]
atleast(3,q([$i.. $j])) ;; au plus 3 parmi toute la liste [q($i)...q($j)]
;; sous l'hypothèse que $i <= $j
```

Les comédiens

Antoine, Bruno, Charles, Denis et Etienne tiennent le rôle d'un roi, d'un soldat, d'un fou, d'un bourreau et d'un prisonnier dans une pièce de théâtre.

- ▶ Antoine, Bruno et le prisonnier ne connaissent pas encore leur texte par coeur.
- ▶ Pendant les pauses, le soldat joue aux cartes avec Denis.
- ▶ Antoine, Bruno et Charles critiquent la manière de jouer du bourreau.
- ▶ Le fou apprécie le jeu de Bruno, de Charles et d'Etienne, mais déteste celui du soldat.

Quel est le rôle de chacun ?

Comédiens en ToulST

```
$roi = 1
$sol = 2
$fou = 3
$bourr = 4
$pri = 5
$a=1
$b=2
$c=3
$d=4
$e=5
not c($a,$pri)
not c($b,$pri)
not c($d,$sol)
not c($a,$bourr)
not c($b,$bourr)
not c($c,$bourr)
not c($b,$fou)
not c($c,$fou)
not c($e,$fou)
not c($b,$sol)
not c($c,$sol)
not c($e,$sol)
bigand $i in [1..5]:
    exact(1,c($i,[1..5]))
end
bigand $i in [1..5]:
    exact(1,c([1..5],$i))
end
```